

Tecnatura Superior en Desarrollo de Software

Módulo 05 — Guards: validación de precondiciones en la capa Application

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Qué es un Guard?

Un **Guard** es un mecanismo de validación que opera como guardián en la entrada de una operación. Su responsabilidad es verificar que los datos que llegan — generalmente desde un DTO o un formulario — cumplen las precondiciones mínimas necesarias para que la operación pueda ejecutarse con seguridad.

La palabra "precondición" es clave: el Guard no valida reglas de negocio — eso es responsabilidad del dominio. El Guard valida que los datos *llegaron bien formados*: que un campo requerido no está vacío, que una cadena no supera el largo máximo permitido, que un Guid no es el Guid vacío, que un email tiene estructura básica de correo. Si alguna de estas condiciones no se cumple, la operación no debe continuar — y el Guard lo expresa acumulando errores descriptivos.

```
Guard          → punto de entrada que recibe un valor y su nombre
GuardBuilder   → acumula reglas encadenadas sobre ese valor
ValidationError → describe un error: qué campo y qué falló
```

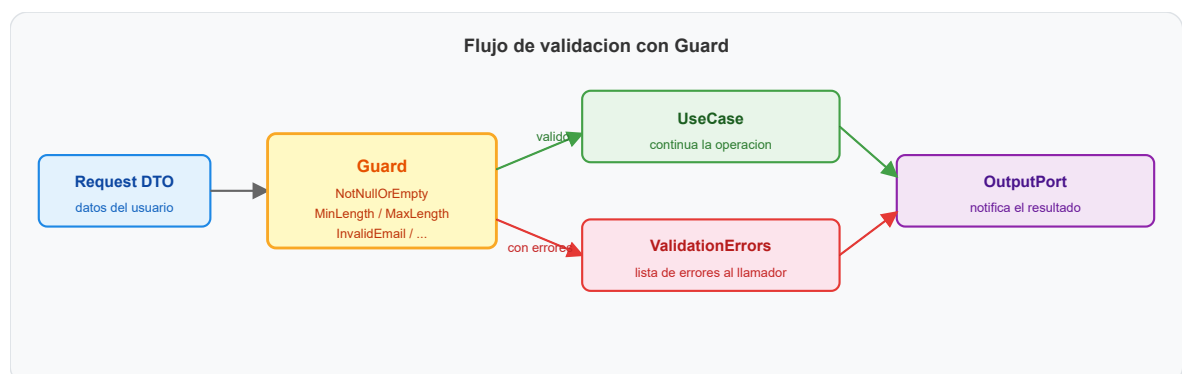
A diferencia de las excepciones — que detienen la ejecución ante el primer problema — los Guards acumulan **todos los errores** de una sola pasada y los devuelven juntos. Esto permite que la interfaz de usuario muestre todos los campos inválidos al mismo tiempo, en lugar de uno por vez.

Analogía del mundo real

Imaginá el control de ingreso a un vuelo internacional. Antes de que el pasajero pase a la sala de embarque, un agente revisa una lista de precondiciones: que el pasaporte no esté vencido, que el nombre coincida con el del ticket, que el equipaje no supere el peso máximo, que tenga la visa correspondiente.

El agente **no decide si el destino es correcto** — eso lo decide la aerolínea. Solo verifica que el pasajero cumple las condiciones mínimas para subir al avión. Si algo falla, anota todos los problemas y se los comunica al pasajero de una sola vez — no uno por vez.

El Guard cumple exactamente ese rol en el software: controla la entrada, acumula todos los problemas encontrados y los informa juntos antes de que la operación comience.



Características esenciales

ValidationError — el portador del error

Antes de definir el Guard, necesitamos el tipo que describe cada error. `ValidationError` es un objeto simple con dos datos: el nombre del campo que falló y el mensaje descriptivo del problema. No es una excepción — es información que se acumula y se devuelve.

```
1 // primary constructor — C# 12
2 public class ValidationError(string propertyName, string message)
```

```
3 {
4     // nombre del campo que falló la validación
5     public string PropertyName => propertyName;
6
7     // mensaje legible para el usuario o la UI
8     public string ErrorMessage => message;
9 }
```

GuardBuilderBase<TBuilder> — la base encadenable

La base de todo Guard es una clase abstracta genérica que implementa el patrón **Fluent Builder**: cada regla devuelve `this` — la misma instancia del builder — para poder encadenar múltiples validaciones sobre el mismo valor en una sola expresión. Los errores se acumulan internamente en una lista.

```
1 // TBuilder es el tipo concreto — permite que cada método devuelva el tipo correcto
2 public abstract class GuardBuilderBase<TBuilder>
3     where TBuilder : GuardBuilderBase<TBuilder>
4 {
5     // lista interna donde se acumulan todos los errores encontrados
6     protected readonly List<ValidationError> _errors = new();
7
8     // nombre del parámetro que se está validando
9     protected readonly string _paramName;
10
11     protected GuardBuilderBase(string paramName) => _paramName = paramName;
12
13     // expone los errores acumulados — solo lectura
14     public IReadOnlyList<ValidationError> Errors => _errors;
15
16     // true si no se encontró ningún error — útil para validar un campo solo
17     public bool IsValid => _errors.Count == 0;
18
19     // método interno para registrar un error con el nombre del parámetro
20     protected void AddError(string message)
21         => _errors.Add(new ValidationError(_paramName, message));
22 }
```

GuardBuilderString — reglas para cadenas de texto

Cada tipo de dato tiene su propio Builder concreto que hereda de `GuardBuilderBase`. `GuardBuilderString` define las reglas aplicables a campos de texto. Cada método verifica una condición, agrega un error si no se cumple y devuelve `this` para permitir el encadenamiento.

```
1 public class GuardBuilderString : GuardBuilderBase<GuardBuilderString>
2 {
```

```
3 // valor que se está validando
4 private readonly string _value;
5
6 public GuardBuilderString(string value, string paramName)
7     : base(paramName) => _value = value;
8
9 // verifica que el valor no sea nulo ni vacío
10 public GuardBuilderString NotNullOrEmpty(string? mensaje = null)
11 {
12     if (string.IsNullOrEmpty(_value))
13         AddError(mensaje ?? $"El campo '{_paramName}' es requerido.");
14     return this;
15 }
16
17 // verifica que la longitud mínima se cumpla
18 public GuardBuilderString MinLength(int min, string? mensaje = null)
19 {
20     if (_value?.Length < min)
21         AddError(mensaje ?? $" '{_paramName}' debe tener al menos {min} caracteres.");
22     return this;
23 }
24
25 // verifica que la longitud máxima no se supere
26 public GuardBuilderString MaxLength(int max, string? mensaje = null)
27 {
28     if (_value?.Length > max)
29         AddError(mensaje ?? $" '{_paramName}' no puede superar {max} caracteres.");
30     return this;
31 }
32
33 // verifica que el valor tenga formato básico de email
34 public GuardBuilderString InvalidEmail(string? mensaje = null)
35 {
36     var regex = new Regex(@"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$");
37     if (string.IsNullOrEmpty(_value) || !regex.IsMatch(_value))
38         AddError(mensaje ?? $" '{_paramName}' no es un correo valido.");
39     return this;
40 }
41 }
```

Guard — el punto de entrada estático

La clase `Guard` es el punto de entrada estático que crea el Builder correspondiente al tipo de dato que se quiere validar. Su método `Against` recibe el valor y el nombre del campo y devuelve el Builder listo para encadenar reglas.

```
1 // clase estática — no se instancia, solo provee el punto de entrada
2 public static class Guard
3 {
```

```
4 // Against – "defender contra" un valor inválido de tipo string
5 public static GuardBuilderString Against(string value, string paramName)
6     => new GuardBuilderString(value, paramName);
7
8 // sobrecarga para enteros
9 public static GuardBuilderInt Against(int value, string paramName)
10     => new GuardBuilderInt(value, paramName);
11
12 // sobrecarga para Guid
13 public static GuardBuilderGuid Against(Guid value, string paramName)
14     => new GuardBuilderGuid(value, paramName);
15 }
```

¿Por qué se usa?

Sin Guards, el código de validación se escribe con bloques `if` dispersos dentro del método que ejecuta la operación. Cada campo requiere su propio bloque, el código crece verticalmente y se vuelve difícil de leer, mantener y extender. Además, con `if` simples, el primer error detiene la validación — el usuario debe corregir campo por campo en lugar de verlos todos juntos.

El Guard resuelve tres problemas a la vez:

1. **Acumulacion:** recorre todos los campos y acumula todos los errores en una sola pasada.
2. **Legibilidad:** cada campo se valida en una línea encadenada — el código expresa la intención claramente.
3. **Reutilización:** las reglas se definen una vez en el Builder y se usan en cualquier UseCase.

Es importante entender qué **no** hace el Guard: no valida reglas de negocio. Si el Guard verifica que el email no está vacío y tiene formato básico, el Value Object `Email` del dominio verificará que cumple las reglas específicas del negocio. Ambos son necesarios y complementarios — no se reemplazan.

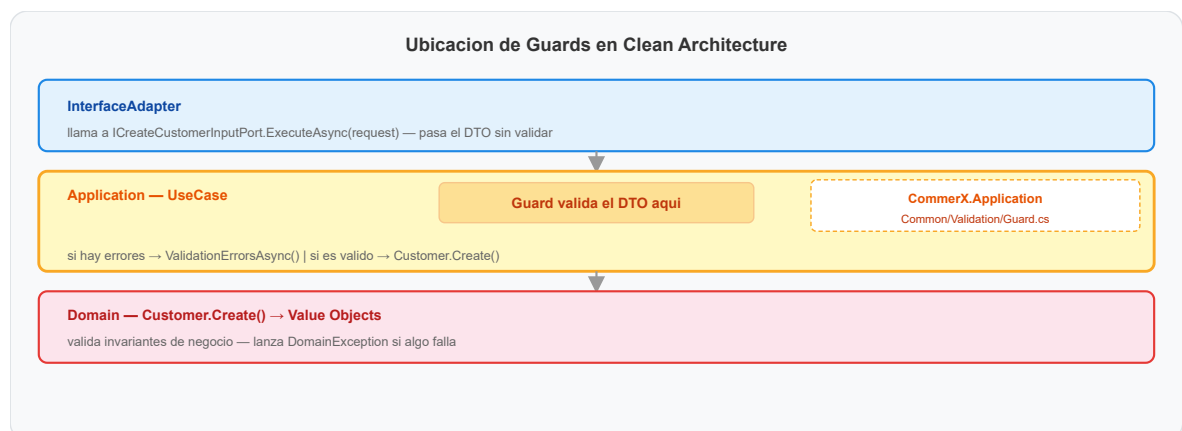
Guards vs. Value Objects — responsabilidades distintas:

Guard — "¿El dato llegó bien formado al sistema?" → precondition tecnica

Value Object — "¿El dato cumple las reglas del negocio?" → invariante de dominio

¿Cómo encaja en la arquitectura?

Los Guards viven en la capa **Application**, en el namespace `Common/Validation/`. No pertenecen al dominio porque no expresan reglas de negocio. No pertenecen a Infrastructure porque no acceden a ningún recurso externo. Son utilidades de la capa de aplicación que protegen la entrada antes de que el dominio sea invocado.



Marco teórico

El patrón Fluent Builder aplicado a la validación

El Fluent Builder es un patrón de diseño que permite construir un objeto complejo paso a paso usando métodos encadenados. En el contexto de los Guards, cada método de validación es un paso que agrega información al resultado final — la lista de errores. La cadena termina cuando se accede a `Errors` o `IsValid`.

```
1 // ejemplo en Main() – validación de un campo de texto
2 string titulo = "";
3
4 // Guard.Against recibe el valor y el nombre del campo
5 var guard = Guard.Against(titulo, "Titulo")
6     .NotNullOrEmpty()
7     .MinLength(3)
8     .MaxLength(100);
9
10 // IsValid es false porque el campo está vacío
11 Console.WriteLine(guard.IsValid); // False
12
13 // recorremos los errores acumulados
14 foreach (var error in guard.Errors)
15     Console.WriteLine($"{error.PropertyName}: {error.ErrorMessage}");
16 // Titulo: El campo 'Titulo' es requerido.
```

Acumulación de errores de múltiples campos

Cuando se validan varios campos, se instancian múltiples Guards y sus errores se combinan en una sola lista. Esto permite enviar al llamador todos los problemas encontrados en una única respuesta.

```
1 // Main() – validación de múltiples campos al mismo tiempo
2 string titulo = "";
3 string autor = "A";
4 string email = "no-es-un-email";
5
6 // validamos cada campo por separado
7 var guardTitulo = Guard.Against(titulo, "Titulo").NotNullOrEmpty().MinLength(3);
8 var guardAutor = Guard.Against(autor, "Autor").NotNullOrEmpty().MinLength(2);
9 var guardEmail = Guard.Against(email, "Email").InvalidEmail();
10
11 // combinamos todos los errores en una sola lista
12 var errors = guardTitulo.Errors
13     .Concat(guardAutor.Errors)
14     .Concat(guardEmail.Errors)
15     .ToList();
16
17 // si hay errores, no continuamos – los mostramos todos juntos
18 if (errors.Count > 0)
19 {
20     foreach (var e in errors)
21         Console.WriteLine($"[{e.PropertyName}] {e.ErrorMessage}");
22     return; // detenemos – no tocamos el dominio
23 }
24
25 // si llegamos aquí, todos los campos son válidos
26 Console.WriteLine("Datos validos – continua la operacion.");
```

Mensajes personalizados

Cada método del Builder acepta un mensaje opcional. Si no se provee, usa un mensaje por defecto generado con el nombre del parámetro. Esto permite adaptar el texto al contexto sin modificar el Builder.

```
1 // mensaje por defecto — usa el nombre del parámetro
2 Guard.Against(isbn, "ISBN").NotNullOrEmpty();
3 // resultado: "El campo 'ISBN' es requerido."
4
5 // mensaje personalizado — más claro para el usuario final
6 Guard.Against(isbn, "ISBN").NotNullOrEmpty("El ISBN del libro es obligatorio.");
7 // resultado: "El ISBN del libro es obligatorio."
```

Errores comunes

Error 1 — validar reglas de negocio en el Guard

El Guard debe verificar precondiciones técnicas — no reglas de negocio. Verificar mayoría de edad, formato específico de DNI o unicidad en la base de datos es responsabilidad del dominio o del repositorio, no del Guard.

ERROR COMUN

Regla de negocio dentro del Guard — rompe la separación de responsabilidades.

```
1 // MAL: mayoría de edad es una regla de negocio
2 Guard.Against(birthDate, "FechaNacimiento")
3     .MustBeMajorOfAge(); // no corr
```

BUENA PRACTICA

El Guard verifica que la fecha llegó — el dominio verifica la edad.

```
1 // Guard: solo verifica que el campo no sea nulo
2 Guard.Against(birthDate, "FechaNacimiento")
3     .NotNull(); // solo precondición
4 // el VO BirthDate valida la mayoría de edad
```

Error 2 — detener en el primer error en lugar de acumular

Usar `if` con `return` ante el primer error impide que el usuario vea todos los problemas a la vez. Los Guards existen precisamente para evitar este patrón.

ERROR COMUN

Validación secuencial con return — el usuario ve un error por vez.

```
1 if (string.IsNullOrEmpty(titulo)) {  
2   if (titulo.Length < 3) { re  
3   if (string.IsNullOrEmpty(autor)) {
```

BUENA PRACTICA

Guard acumula todos los errores — el usuario los ve todos juntos.

```
1 var g1 = Guard.Against(titulo, "Tit  
2 var g2 = Guard.Against(autor, "Auto  
3 var errors = g1.Errors.Concat(g2.Er
```

Error 3 — ignorar los errores del Guard y continuar

Crear un Guard y no verificar `IsValid` o `Errors` antes de continuar hace que la validación no tenga ningún efecto. Es el equivalente a instalar una puerta de seguridad y no cerrarla.

ERROR COMUN

Guard ejecutado pero sus errores nunca se verifican.

```
1 Guard.Against(titulo, "Titulo").Not  
2 // continúa sin verificar - validac  
3 ContinuarOperacion(titulo);
```

BUENA PRACTICA

Siempre verificar `errors.Count` antes de continuar.

```
1 var guard = Guard.Against(titulo, "  
2 if (!guard.IsValid) return;  
3 ContinuarOperacion(titulo);
```

Qué NO hacer**No duplicar lógica entre Guard y Value Object**

Si el Guard y el Value Object validan lo mismo — por ejemplo ambos verifican el formato del email — la lógica está duplicada en dos capas distintas. Esto genera inconsistencias cuando una regla cambia y se actualiza en un lugar pero no en el otro.

QUE NO HACER

Guard y VO validan el mismo formato — lógica duplicada.

```
1 // Guard valida formato de email
2 Guard.Against(email, "Email").Invalid
3
4 // VO Email también valida el mismo
5 // → si cambia la regex, hay que ac
```

BUENA PRACTICA

Guard verifica que no está vacío. El VO valida el formato.

```
1 // Guard: precondition – campo no v
2 Guard.Against(email, "Email").NotNu
3
4 // VO Email: invariante de negocio
5 // cada uno tiene su responsabilidad
```

No usar Guards en el dominio ni en Infrastructure**QUE NO HACER**

Guard dentro de un Value Object o repositorio — capa incorrecta.

```
1 // MAL: Guard en un Value Object de
2 public sealed record BookTitle : Va
3 {
4     public static BookTitle Create(
5     {
6         // Guard en Domain – no cor
7         Guard.Against(value, "title
8     }
9 }
```

BUENA PRACTICA

El VO usa su propia lógica y lanza DomainException — sin Guard.

```
1 public sealed record BookTitle : V
2 {
3     public static BookTitle Create
4     {
5         // lanza su excepción de d
6         if (string.IsNullOrEmptySp
7             throw new InvalidBookT
8         return new BookTitle(value
9     }
10 }
```